

Enhancing Computational Efficiency: Transitioning from Serial to Parallel Programming for Low-to-High Resolution Video Reconstruction Using FSRCNN on AMP Architecture

Annisa

Department of Informatics
Hasanuddin University, Indonesia
annisa21d@student.unhas.ac.id

Adnan

Department of Informatics
Hasanuddin University, Indonesia
adnan@unhas.ac.id

Zahir Zainuddin

Department of Informatics
Hasanuddin University, Indonesia
zahir@unhas.ac.id

Abstract— Super Resolution (SR) video has become a major concern in the image processing community, as increasing video resolution from low to high still faces various challenges. This research aims to improve the performance of the FSRCNN program on asymmetric multicore processors by leveraging the advantages of the big.LITTLE architecture. By converting a serial program to parallel, processors can operate and access shared data simultaneously, thereby potentially increasing the computational speed that is often bottlenecked at certain stages in the program, which can also affect video pixel accuracy. The results showed that the parallel scenario using 4 big CPUs produced the best video quality with the fastest computation time compared to the other scenarios. In contrast, the worst scenario occurs when using 4 LITTLE CPUs, which results in the longest computation time and lower video quality with the appearance of more artifacts. Meanwhile, the hybrid scenario (big.LITTLE and LITTLE.big, 8 threads) offers a balance between performance and video quality, although it still cannot rival the results obtained with big CPUs. The main factors affecting the computation time in FSRCNN are the use of LITTLE CPUs as well as the processing bottleneck at layer 8 (deconvolution stage), which shows the least speedup in the parallel scenario.

Keywords— FSRCNN, Parallel Programming, High Resolution, AMP.

I. INTRODUCTION

Super Resolution (SR) is a technique designed to enhance the resolution of videos or images, enabling the production of clearer and more detailed visuals. However, the implementation of super-resolution faces several challenges, including degraded dataset quality caused by data duplication, inadequate image resolution, and overall low data quality. To address these issues, various advanced techniques have been developed, with FSRCNN (Faster Super Resolution Convolutional Neural Network) standing out as a highly efficient method for improving image resolution and quality [1][2][3].

Intel has introduced two generations of asymmetric multicore processors (AMPs) [4]. AMPs integrate two different type of core processor, the “big” processor are designed to optimize single-threaded performance by increasing clock frequency and reducing latency through simultaneous multithreading. Meanwhile, “LITTLE” processors work to prioritize energy efficiency in multithreaded programming. Both types of processors use the same instruction set, allowing them to run multithreaded processes simultaneously [5].

In the field of computing technology, AMP architectures have emerged as a major focus of research [6], [7], [8]. To

harness the potential of AMPs, good scheduling decisions are essential although they can deftly manage certain aspects of the system’s asymmetric processing, they fail to address all of the runtime variables that affect AMPs, particularly in scenarios involving common multithreaded or multi programmed workloads [7].

LITTLE CPUs in AMPs pose significant challenges to the runtime system of parallel programming models. While previous research has focused on optimizing the execution of task-based parallel applications on AMPs through improvements in the runtime system, improving the performance of unmodified data parallel applications on this architecture remains a major challenge. This is largely due to workload imbalance, which can lead to increased program runtimes [9].

In this study, FSRCNN was selected for its exceptional ability to accelerate the image reconstruction process compared to other SR methods, making it highly suitable for evaluation within the context of AMP architecture. This research focuses on analyzing the impact of race conditions on FSRCNN programs running on AMP devices. The process of enhancing low-resolution images to high-resolution is carried out by parallelizing the program using OpenMP, followed by an analysis of the big.LITTLE architecture to identify and understand race conditions within the program.

II. RELATED WORKS

Research [10] reports on models in system performance and parallelization related to acceleration and energy consumption, specifically three classical acceleration models Amdahl’s law, Gustafson’s model, which are extended to cover a range of heterogeneous system configurations that can be described as normal-form heterogeneity. The work in this study is validated through experiments conducted using the big.LITTLE heterogeneous platform.

Research [11] analyzes the utilization of loop-based OpenMP as a method to evaluate performance on multicore systems, where loop scheduling cannot effectively address workload imbalances caused by performance differences exhibited by heterogeneous processors or Asymmetric Multi Processors (AMPs). This research identifies that although loop-based OpenMP is designed to improve efficiency in parallel processing, traditional loop scheduling often cannot handle performance variability among cores that have different characteristics.

As a result, the workload may not be evenly distributed, leading to a decrease in overall performance. This research highlights the need for alternative approaches to scheduling and workload distribution that are more adaptive to performance differences in multicore systems, especially in configurations with AMPs, to maximize efficiency and reduce the negative impact of workload imbalance.

The authors of [12] presented research on performance evaluation of heterogeneous multicore processors in the context of multi-threaded workloads. In the paper, they showed that heterogeneous multicore processors are more efficient compared to homogeneous multicore processors when handling multithreaded workloads. The authors used a weighted speed-based efficiency metric to support their findings, which is calculated by dividing the total IPC (Instructions Per Cycle) of all running threads by the total IPC of the slowest single core.

After studying the Fast Super-Resolution Convolutional Neural Network (FSRCNN) program, which includes convolution and deconvolution stages [13] [14] [15], researchers identified an opportunity to improve computational performance by switching from serial to parallel execution. They proposed a new parallel programming strategy aimed at improving the program's efficiency. The parallel implementation of the FSRCNN algorithm allows for the simultaneous execution of its steps, optimizing the utilization of computational resources. This research contributes to the trend of optimizing super-resolution techniques with deep learning, opening up new potential in high-resolution image processing.

In the research conducted by Chaitanya, that in this study more than 90 percent of the computation time is spent in the convolution stage, so the convolution filter operation is parallelized in the CNN model using a new GPU optimization method. But in this study using the power of the GPU only parallelizes the Convolution part so that the entire Convolution operation is performed in parallel and the computation time is reduced from 450 seconds to 2 seconds per frame for 25 Output Frames [16].

This research aims to accelerate the FSRCNN program by converting the serial program[17] into a parallel program to improve its performance, by looking at the effect of race conditions on video image results. We use a parallelization approach on the previously serialized program, allowing processors to operate simultaneously and thus improving the computational efficiency of the program

III. MATERIAL AND EXPERIMENTAL CONFIGURATION

A. Orange Pi 5

The Orange Pi 5 features the Rockchip RK3588S, a next-generation octa-core 64-bit ARM processor that combines quad-core Cortex-A76 and quad-core Cortex-A55 cores. It utilizes Samsung's 8nm LP process technology, with the large cores reaching up to 2.4 GHz. The processor also integrates an ARM Mali-G610 MP4 GPU, equipped with high-performance 3D and 2D image acceleration modules.

The Orange Pi 5 provides a wide range of interfaces, including HDMI output, a Type-C port, M.2 PCIe 2.0 x1, a Gigabit Ethernet port, USB 2.0, USB 3.0 interfaces, and a 26-pin expansion header. This makes it versatile for applications in high-end tablets, edge computing, artificial intelligence,

cloud computing, AR/VR, smart security, smart homes, and other fields, covering a broad spectrum of AIoT industries.

B. Software setup and configuration

Table 1 shows, The use of software configurations such as-fopenmp and-O3 in GCC is an effective strategy to optimize program performance. -fopenmp enables parallel programming that makes use of multiple CPU cores, while-O3 enables various optimization techniques to increase program execution speed. The combination of these two options can help you get the best performance out of your application, especially if you are working with applications that require a lot of calculations or data processing.

TABLE I. SOFTWARE SETUP AND CONFIGURATION.

Software	Version	Compiler flags
Parallel Compiler	gcc version 11.3.0	gcc, -fopenmp, -O3
Serial Compiler	gcc version 11.3.0	gcc, -O3

Table 2 shows, the performance power in the AMPs architecture various configurations are shown. Keeping performance and power as the main concerns, we implement four different configurations.

TABLE II. PERFORMANCE SETUP

Config Index	CPU	Frequency	Type
Configuration 1	0-3	1.800 MHz	L CPU
Configuration 2	4-7	2.400 MHz	B CPU
Configuration 3	7-0	2.400 -1.800 MHz	B+L CPU
Configuration 4	0-7	1.800 -2.400 MHz	L+B CPU

Table 3 shows, several experiments were conducted in various execution scenarios. The scenarios were designed to show the difference in scalability between using L-CPU, B-CPU, Hybrid B+L CPUs, and Hybrid L+B CPUs. L CPUs is set to work with processor (0, 1, 2, 3), B-CPU is set to work with processor (4, 5, 6, 7), Hybrid B+L CPUs is set to work with processor (4, 5, 6, 7, 0, 1, 2, 3), and Hybrid L+B CPUs is set to work with processor (0, 1, 2, 3, 4, 5, 6, 7).

TABLE III. PARALLEL SCHEDULING SCENARIO

scenario	#threads	CPU Set mask
big_CPU	0-3	{4,5,6,7}
Little_CPU	4-7	{0,1,2,3}
big+Little CPU	7-0	{4,5,6,7,0,1,2,3}
Little+big CPU	0-7	{0,1,2,3,4,5,6,7}

In the first scenario, all jobs are placed on the L CPUs, which are optimized for efficiency and power-saving. In the second scenario, all jobs are placed on the B CPUs, which are designed for high performance and can handle intensive workloads more effectively. In the third scenario, jobs are distributed between B-CPU and L-CPU (Hybrid B+L), leveraging the strengths of both types of processors to achieve a balance between performance and power efficiency. In the fourth scenario, jobs are distributed between L-CPU and B-CPU (Hybrid L+B), aiming to optimize performance while managing power consumption effectively. Additionally, for serial jobs, there are two distinct scenarios: one utilizing a B-CPU and another using an L-CPU. This allows for a comparative analysis of how each type of CPU handles individual tasks in a serial execution context.

C. Parameter Usage

Switching from serial programming to parallel programming can significantly reduce computation time by utilizing multiple processors or cores to execute tasks

simultaneously. In the serial approach, execution is done sequentially by working on tasks one by one, which means the total time increases with complexity and large data size. In contrast, in parallel programming, parts of a task can be processed simultaneously, resulting in increased speed and efficiency. However, the success of parallelization also depends on the characteristics of the task at hand and how overheads that may arise from sharing and managing work between processors simultaneously can affect the total computation time. In this case, we use a formula to calculate the speedup in the performance process of a program, which involves (T_s), the execution time in the serial model, and (T_p), the execution time in the parallel model. By calculating the speedup as (1).

$$\text{SpeedUp (p)} = T_s/T_p \quad (1)$$

This research evaluates the effectiveness of parallel programming in reducing computation time compared to the serial approach. The data collected provides detailed insights into the potential improvements in video resolution while preserving image quality, using different CPU configurations in a parallel computing environment.

D. Parameter Usage

Low-resolution video refers to video with a lower level of resolution, resulting in limited detail and less visual sharpness. Video resolution is usually measured in pixels and is expressed as pixel width x pixel height.

In Table 4, in this scenario, the low-resolution video input has a size of 176x144 pixels, with a total of 25,344 pixels. After converting to a size of 352x288 pixels, the total pixels increase to 101,376. This increase in the number of pixels can potentially improve the overall image quality, allowing for a clearer view and better detail in the displayed video.

TABLE IV. INFORMATION DATA

	Input Size	Output Size	Nr Frames
Suzie	176x 144	352x288	150

An increase in the number of pixels can occur due to various factors such as poor transmission or storage, compression, low initial image capture, and so on. In this scenario, video resolution may be degraded due to some of the mentioned factors. We obtained the low video resolution data from <https://engineering.purdue.edu/Engr>.

The transformation from serial to parallel execution in FSRCNN follows a systematic series of stages. The first stage involves code analysis to identify computational blocks that can be parallelized. This process focuses on code sections with intensive looping and evaluates whether calculations within each iteration are independent or have data dependencies on previous iterations. If the calculations within the loop are independent, the code block can be parallelized using OpenMP to efficiently distribute workloads across multiple threads. In addition to implementing OpenMP, set affinity is utilized to control processor selection for executing computational tasks based on predefined test scenarios. Set affinity enables precise thread scheduling on specific processor types, ensuring that each scenario runs with an optimal configuration.

By employing this approach, the study conducts a comprehensive evaluation of workload distribution on asymmetric processors. The results provide valuable insights into how different workload distributions across the

big.LITTLE architecture influence execution performance, image quality, and overall computational efficiency in the FSRCNN program.

IV. EXPERIMENTAL RESULTS

fig.1 in the FSRCNN processing program shows significant results regarding computing time in various processing scenarios. with the parallel scheduling scenario, it can be seen that the longest computing time occurs in the scenario using 4 threads from LITTLE CPUs, which takes 75 seconds, while it can be seen that the fastest computing time is achieved in the scenario using 4 threads from big CPUs, which only takes 24.3 seconds. This advantage emphasizes the efficiency of data processing when multiple threads are used simultaneously.

Furthermore, the hybrid configuration combining 8 threads of big.LITTLE CPUs showed slightly slower performance, with a computing time of 26 seconds. with a very significant time difference between the use of the 4 big CPUs and 4 LITTLE CPUs scenarios, it confirms that using LITTLE CPUs which are power efficient will cause delays in data processing. While the time difference between the scenario of using 4 big CPUs and Hybrid is only 3 seconds, this still reflects the advantage of the parallel approach by using only 4 big CPUs more profitable. In contrast, the serial scheduling scenario using a LITTLE CPU experienced the longest computing time, namely 169.5 seconds, which highlights how inefficient resource use is in non-parallel processes. These findings demonstrate the importance of selecting appropriate computing strategies to improve data processing performance.

The parallel scenario using 4 LITTLE CPUs showed the worst performance in FSRCNN processing, with longer computation time and lower video quality. This is due to the limited computing power of LITTLE CPUs, which have lower clock speeds and an architecture optimized for power efficiency, rather than for heavy processing tasks such as high-resolution video reconstruction. In addition, the lower memory bandwidth on LITTLE CPUs leads to slower data access, thus worsening performance. Overhead in thread synchronization is also a factor that slows down processing, where inter-thread coordination running on LITTLE processors is not as efficient as on big CPUs. In addition, a higher probability of race conditions can lead to data mismatches, resulting in degradation of the resulting image quality.

TABLE V. TIME RESULTS

	Scenario	threads	Time	SpeedUp
Serial	big CPU	1	86s	
Serial	Little CPU	1	169s	
Parallel	big CPUs	4	24s	+3
Parallel	Little CPUs	4	75s	2
Parallel	big+Little CPUs	8	26s	3
Parallel	Little+big CPUs	8	26s	3

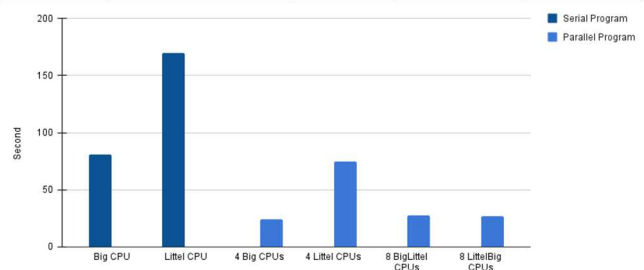


Fig. 1. Execution Time in Serial and Parallel Programs.

The large amount of execution time spent on serial programs is a challenge in this study. We converted a serial program into a parallel program and examined the effect of race conditions on the FSRCNN program when it was compiled in parallel and analyzed its impact on processor usage. Race conditions in certain sections occurred when the program was run using parallel programming techniques. Although some scenarios resulted in good and acceptable video quality, there were also scenarios where many pixel values were corrupted due to race conditions, resulting in poor video quality.

Race conditions in the FSRCNN parallelization process occur when multiple threads simultaneously access and modify pixel data without proper synchronization, leading to inconsistencies in video reconstruction. In this context, race conditions can cause color mismatches, visual artifacts, or loss of detail in certain pixels, ultimately affecting the final video quality. This issue is most prevalent during the deconvolution stage (layer 8), where each generated pixel depends on computations from surrounding pixels. If multiple threads operate on adjacent areas without proper coordination, the calculated pixel values may become inaccurate or inconsistent.

Race conditions are more frequent when using LITTLE CPUs, which have limited processing power and lower memory bandwidth compared to big CPUs. These limitations contribute to increased pixel miscalculations, degrading the overall video quality. However, when utilizing big CPUs, the reconstructed video exhibits greater stability and higher quality due to their superior processing capacity, which enables more efficient task scheduling and minimizes the negative impact of race conditions.

In parallel execution, utilizing 4 threads on big CPUs produced the best image quality, closely resembling the reference image with minimal distortion. Conversely, processing with 4 threads on LITTLE CPUs resulted in the lowest image quality, exhibiting noticeable artifacts and distortions due to the limited computational capacity of LITTLE CPUs. In hybrid configurations with 8 threads, both big.LITTLE and LITTLE.big setups provided image quality that was superior to the LITTLE CPU scenario and approached the performance of the big CPU configuration, demonstrating a balanced trade-off between performance and visual accuracy.

It can be seen in Fig.2 parts (a) and (b) that when the program is run serially, the overall video frames produced have no artifacts because the program executes the data one by one, so there are no race conditions that can corrupt the pixel data. In contrast, in the parallel program, Part (c) with the use of 4 big CPUs shows a three times better improvement in computation time compared to the serial program, and produces good video frame quality. However, in Part (d) with the use of 4 LITTLE CPUs, the computation time becomes slow and produces many artifacts and video pixel damage. These corruptions and time delays occur because the architecture of LITTLE CPUs designed for power efficiency may cause race conditions when the program is executed in parallel mode. This is in contrast to the use of 4 big CPUs that can improve computation time and produce video frames that are almost as good in quality as the serial mode. Parts (e) and (f) show almost the same quality results as (c), but with more degraded performance.

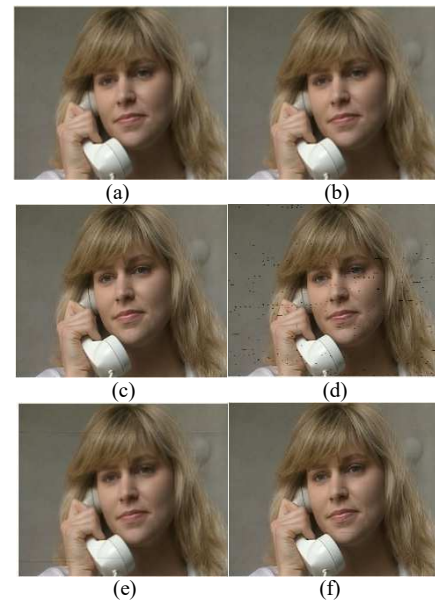


Fig. 2. Frame Results

Overall, these results confirm that utilizing big CPUs in parallel computing produces higher image quality, whereas using LITTLE CPUs tends to introduce more distortion.

The results presented in Fig. 3, the serial program run shows that the layer that consumes the most computing time in the FSRCNN program in serial mode is layer 8, this happens with the use of big CPU and the use of LITTLE CPU. But overall in the serial program the use of LITTLE CPU will consume the most computing time compared to the use of big CPU. Serial scheduling using LITTLE CPU shows that layer 8 is the most computationally time-consuming part of the program. In contrast, using big CPU in serial scheduling results in better performance, contributing to the overall improvement of program performance.

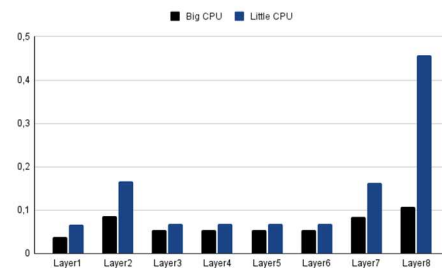


Fig. 3. Serial Execution Time in multiple layers

The results presented in Fig. 4 parallel scheduling scenario, the most significant computation time issues also occur at layer 8. The longest computation time is observed when using 4 threads on a LITTLE CPUs, while the fastest computation time is achieved using the 4 thread approach on big CPUs. SpeedUp results from using the 4 Big CPUs scenario show a 3x faster improvement compared to the serial program. In FSRCNN, deconvolution (layer 8) is the final stage responsible for converting a low-resolution (LR) image into a high-resolution (HR) image. This process requires extensive computation, as each output pixel is derived from a combination of multiple pixels in the previous layer. Consequently, this stage is prone to race conditions in parallel computing, especially when CPU resources are limited, such as when using LITTLE CPUs, which have lower performance compared to big CPUs.

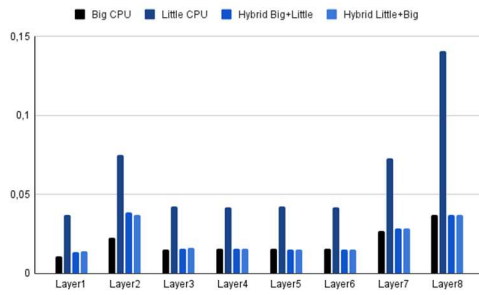


Fig. 4. Parallel Execution Time in multiple layers

Overall, Fig.5 present the results of the big+LITTLE and LITTLE+big hybrid scenarios using 8 threads. The increased computation time in the FSRCNN program under the hybrid scenario is further exacerbated by the limited performance of the LITTLE CPU. However, in both scenarios, layer 8 exhibits the lowest SpeedUp during execution.

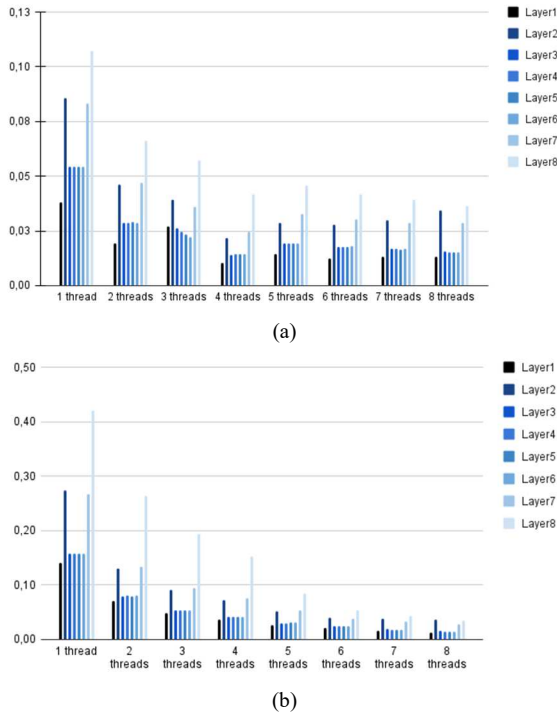


Fig. 5. Execution Time in multiple thread scheduling (a) big.LITTLE Scenarios (b) LITTLE.big scenarios

Fig.6 illustrate the SpeedUp results for different processing scenarios. When using 4 big CPUs, layers 1 to 7 achieve up to a 3x SpeedU, demonstrating efficient parallel execution. However, in layer 8, there is a noticeable decline in SpeedUp, likely due to increased computational complexity and memory dependencies in the deconvolution process. This bottleneck suggests that layer 8 requires additional optimization strategies, such as improved memory access patterns or workload balancing, to enhance parallel efficiency. Furthermore, the disparity in SpeedUp between earlier layers and layer 8 highlights the challenges of scaling performance in deep learning-based image reconstruction tasks.

In contrast, the scenario using 4 LITTLE CPUs achieves approximately 1x SpeedUp for layers 1 to 7 compared to the serial execution, indicating minimal performance gains due to the lower processing power of LITTLE CPUs. Interestingly, for layer 8, both big and LITTLE CPUs exhibit an approximate 3x SpeedUp compared to the serial program.

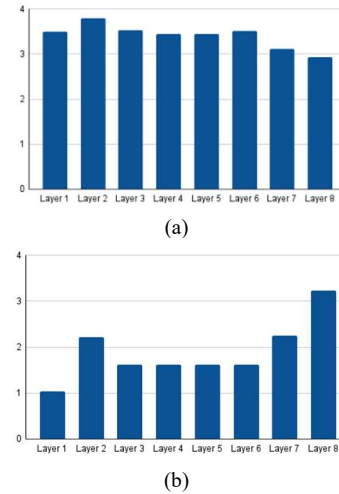


Fig. 6. SpeedUp of each layer (a) 4 big CPUs (b) LITTLE CPUs

Fig.7 show that the Hybrid 8 big+LITTLE CPUs scenario is not much different from the 8 LITTLE+big CPUs scenario. This indicates that the advantages of parallel execution can still be obtained, with 3x better improvement compared to serial programs. These results show that using a combination of big and LITTLE CPUs is also effective in improving application performance through parallel execution.

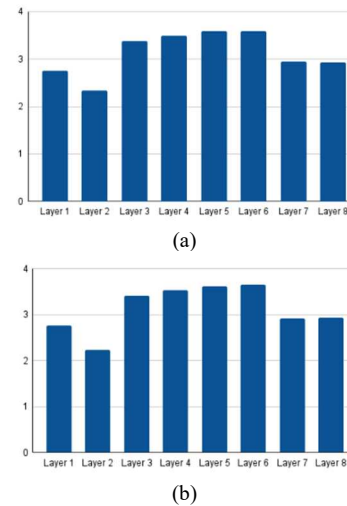
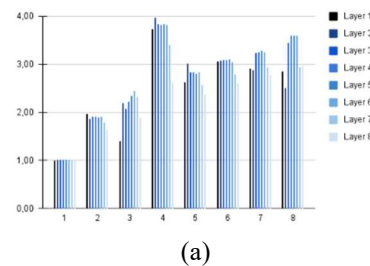


Fig. 7. SpeedUp of each layer (a) Hybrid 8 big.LITTLE CPUs (b) Hybrid 8 LITTLE.big CPUs

Fig.8 detail the SpeedUp results of the big.LITTLE and LITTLE+big hybrid configurations, with an incremental increase in the number of threads, from 1 thread to 8 threads. This experiment aims to demonstrate the performance changes that occur as the number of threads increases in both hybrid scenarios. The results show that both hybrid scenarios achieve nearly the same SpeedUp value, approximately 3 times faster than the serial program.



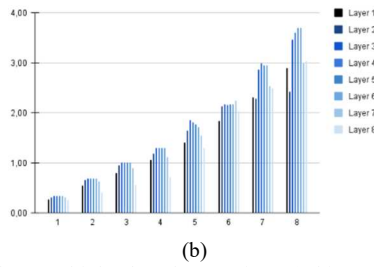


Fig.8. SpeedUp multiple thread scenarios (a) big.LITTLE CPUs (b) LITTLE.big CPUs

The results displayed in Fig.9 show that the highest SpeedUp is achieved in the parallel scenario with 4 big CPU threads. This scenario demonstrates a significant performance improvement compared to the other configurations. It is followed by the hybrid scenario configuration. Meanwhile, the lowest SpeedUp occurs in the scheduling scenario with the use of 4 LITTLE CPUs, which experienced a performance decrease due to their limited processing capabilities compared to the big CPUs. This indicates that higher-performance cores contribute significantly to accelerating the FSRCNN process, while the hybrid approach offers a balance between efficiency and resource utilization. The findings also suggest that the effectiveness of parallelization depends not only on the number of threads but also on the processing power of the assigned cores, emphasizing the importance of task distribution in heterogeneous computing environments.

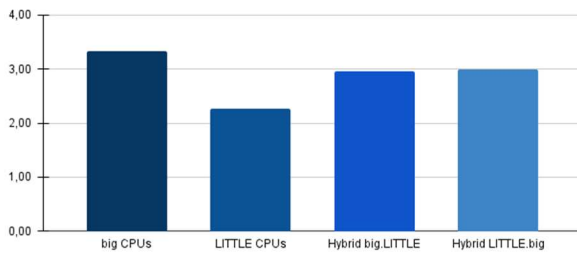


Fig. 9 SpeedUp in multiple thread scheduling (LITTLE.big scenarios)

V. CONCLUSIONS

This research demonstrates that parallelization in FSRCNN with big.LITTLE thread scheduling significantly impacts both video quality and computation time. Experimental results indicate that utilizing 4 big CPUs achieves the best video quality and the fastest execution time of 24 seconds, yielding a 3× speedup compared to the serial mode on big CPUs, which requires 86 seconds. In contrast, executing FSRCNN with 4 LITTLE CPUs results in a significantly longer execution time of 75 seconds, achieving only a 2× speedup compared to the serial mode on LITTLE CPUs, which takes 169 seconds. This scenario also leads to an increase in visual artifacts due to the lower processing capabilities of LITTLE CPUs.

The hybrid scheduling scenarios, which utilize a combination of big+LITTLE and LITTLE+big configurations with 8 threads, demonstrate a balanced performance with an execution time of 26 seconds and a speedup of approximately 3×. Although this configuration improves efficiency compared to the LITTLE CPU scenario, it still falls short of the performance achieved by the 4 big CPUs scenario.

Furthermore, analysis of individual processing layers reveals that the deconvolution stage (layer 8) becomes a major bottleneck in the parallel execution of FSRCNN, causing a noticeable reduction in SpeedUp due to increased computational complexity and memory dependencies. These findings highlight the critical role of thread scheduling strategies and synchronization mechanisms in optimizing parallel computing performance. Therefore, future research should explore alternative parallelization strategies, workload distribution techniques, and hardware configurations to further enhance the efficiency and accuracy of video super-resolution processing using FSRCNN.

REFERENCES

- [1] T. Khachatryan, D. Galstyan, and E. Harutyunyan, "A comprehensive approach for enhancing deep learning datasets quality using combined ssim algorithm and fsrnn," in 2023 IEEE East-West Design Test Symposium (EWDTS), 2023, pp. 1–4.
- [2] J. Zhang and D. Huang, "Image super-resolution reconstruction algorithm based on fsrnn and residual network," in 2019 IEEE 4th International Conference on Image, Vision and Computing (ICIVC), 2019, pp. 241–246.
- [3] C. Dong, C. C. Loy, K. He, and X. Tang, "Image super-resolution using deep convolutional networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 2, pp. 295–307, 2016.
- [4] H.-D. Cho, P. D. P. Engineer, K. Chung, and T. Kim, "Benefits of the big. little architecture," *EETimes*, Feb, 2012.
- [5] Adnan, "Performance evaluation on work-stealing featured parallel programs on asymmetric performance multicore processors," *Array*, vol. 19, p. 100311, 2023.
- [6] M. Dadi, S. Pandey, A. Behera, and T. Venkatesh, "Performance study of partitioned caches in asymmetric multi-core processors," *arXiv preprint arXiv:2304.05442*, 2023.
- [7] T. Yu, R. Zhong, V. Janjic, P. Petoumenos, J. Zhai, H. Leather, and J. Thomson, "Collaborative heterogeneity-aware os scheduler for asymmetric multicore processors," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 5, pp. 1224–1237, 2020.
- [8] J. C. Saez, F. Castro, and M. Prieto-Matias, "Enabling performance portability of data-parallel openmp applications on asymmetric multicore processors," in *Proceedings of the 49th International Conference on Parallel Processing*, 2020, pp. 1–11.
- [9] O. J. On and F. A. B. Hussin, "Evaluation and performance analysis of heterogeneous multicore cluster processor architecture," 2014.
- [10] A. Rafiev, M. A. Al-Hayanni, F. Xia, R. Shafik, A. Romanovsky, and Yakovlev, "Speedup and power scaling models for heterogeneous many-core systems," *IEEE Transactions on Multi-scale computing systems*, vol. 4, no. 3, pp. 436–449, 2018.
- [11] M. Wang, S. Ding, T. Cao, Y. Liu, and F. Xu, "Asymo: scalable and efficient deep-learning inference on asymmetric mobile cpus," in *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, 2021, pp. 215–228.
- [12] V. Mane, S. Jadhav, and P. Lal, "Image super-resolution for mri images using 3d faster super-resolution convolutional neural network architecture," in *ITM Web of Conferences*, vol. 32. EDP Sciences, 2020, p. 03044.
- [13] C. Dong, C. C. Loy, and X. Tang, "Accelerating the super-resolution convolutional neural network," in *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part II 14*. Springer, 2016, pp. 391–407.
- [14] Y. Cao, C. Wang, C. Song, Y. Tang, and H. Li, "Real-time super-resolution system of 4k-video based on deep learning," in 2021 IEEE 32nd International Conference on Application-specific Systems, Architectures and Processors (ASAP). IEEE, 2021, pp. 69–76.
- [15] K. C. P. Tanay, S. Khanna, V. Chandrasekaran, and P. Baruah, "Fast video super resolution using deep convolutional networks," in 2017 International Conference on Innovations in Information, Embedded and Communication Systems (ICIIECS). IEEE, 2017, pp. 1–6.
- [16] M. Abdollahzadeh, "Implementing Deep Convolutional Neural Network in C without External Libraries for YUV video Super-Resolution." github, 2021